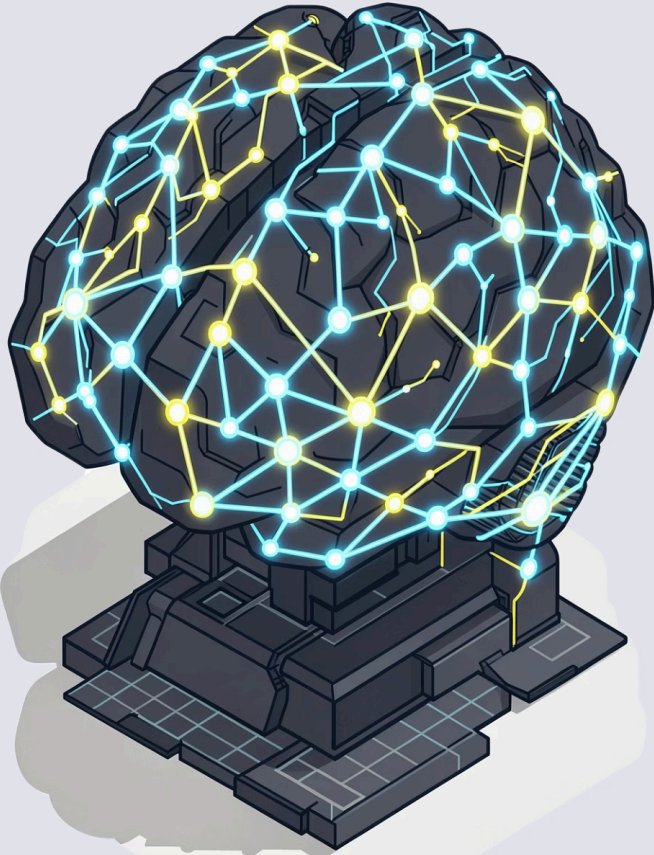




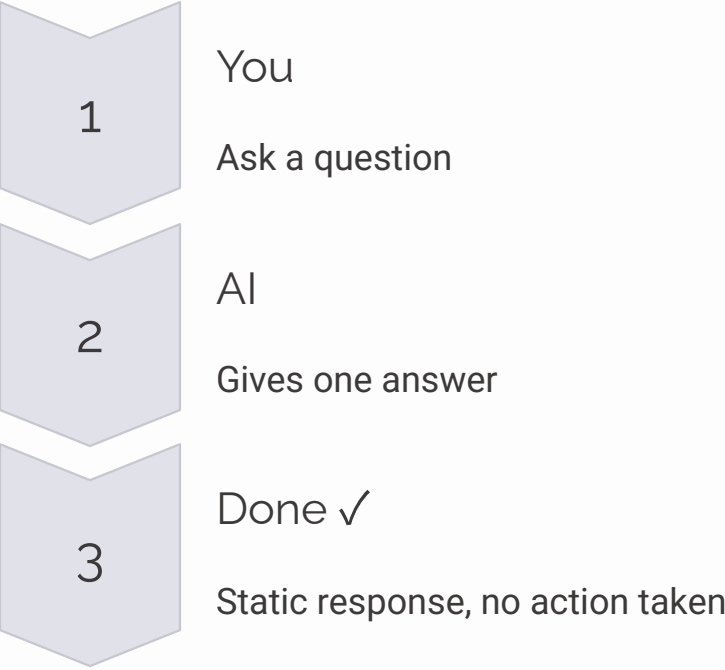
AI Agent Frameworks: A Beginner's Guide

LangChain · AutoGPT · CrewAI · Memory · Planning & Reasoning

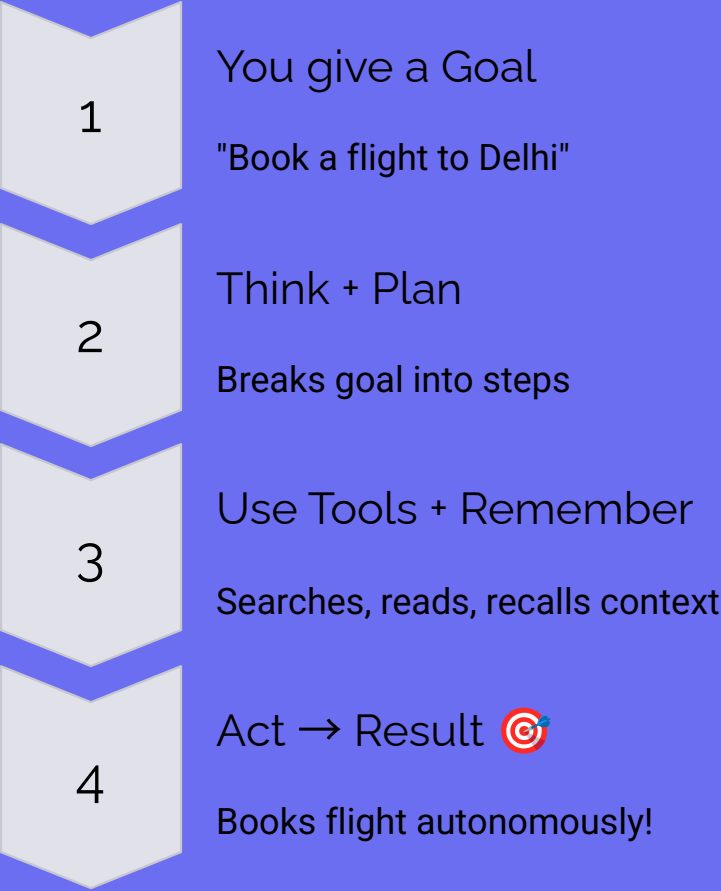
Dr. Anilkumar V. Brahmane | Computer Engineering, Sanjivani College of Engineering

What is an AI Agent? The Big Picture

NORMAL CHATGPT



AI AGENT



🏨 Real Example: Hotel Search in Singapore

Prompt: "Find the best hotels in Singapore for my conference and prepare a comparison."

BASIC LLM

Lists a few hotel names from training data
– **no real search, no comparison, no memory.**

⚠️ Static, outdated, no action taken

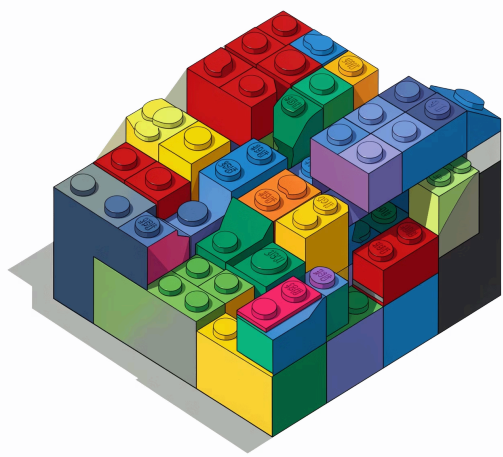
AI AGENT – 7 STEPS

01	02
Understand Goal	Search Internet
Conference hotel near venues	Live hotel listings & reviews
03	04
Collect Hotel Info	Compare Prices & Locations
Rates, locations, amenities	Proximity to conference venue
05	06
Remember Budget Preference	Prepare Comparison Table
Uses your stated budget from memory	Structured, easy-to-read output
07	
Give Recommendation	
Best fit with reasons – done! ✅	

🔑 The Formula

🧠 LLM Reasoning core	🔧 Tools Search, code, files
💾 Memory Remembers context	📋 Planning Breaks into steps
🤔 Reasoning Decides what to do	⚡ Actions Executes real tasks

AI Agent = LLM + Tools + Memory + Planning + Reasoning + Actions

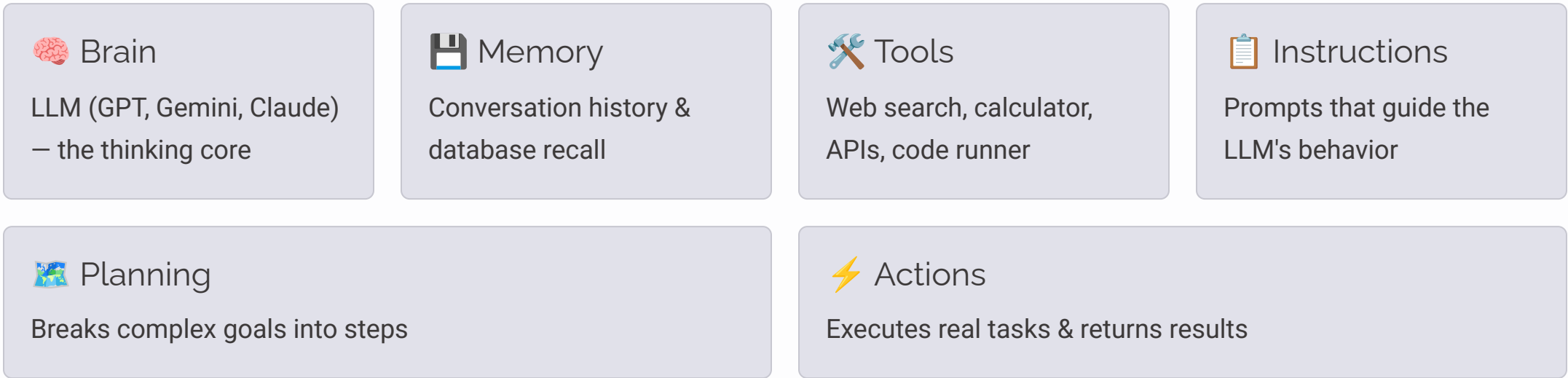


What is LangChain?

LangChain is a **Python framework** — a **toolbox** that connects LLMs with tools, memory, and data sources to build powerful AI applications. Like **LEGO bricks**, you snap ready-made components together!

LangChain Toolbox — Building a Robot

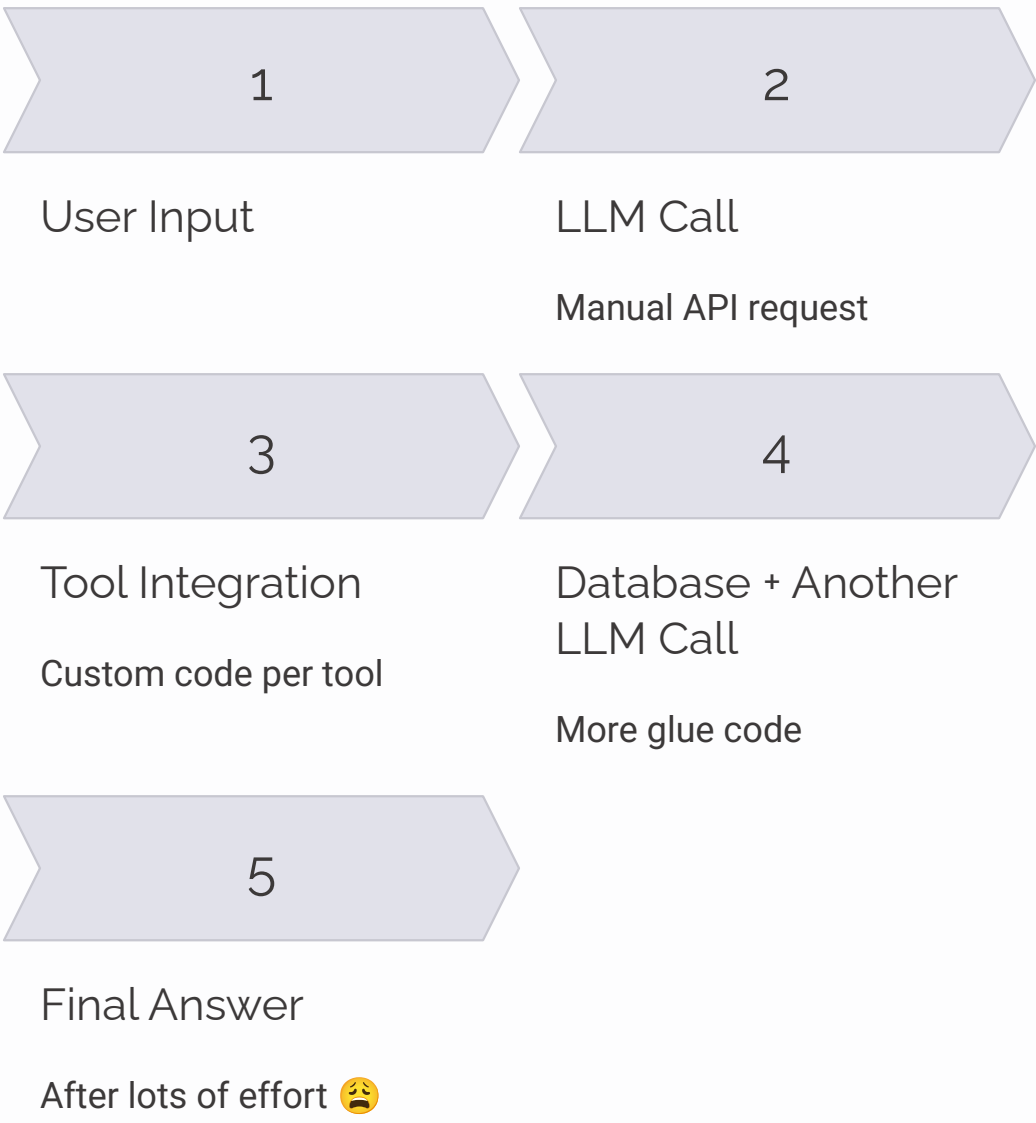
To build a smart AI application, you need all these parts:



Without LangChain vs. With LangChain

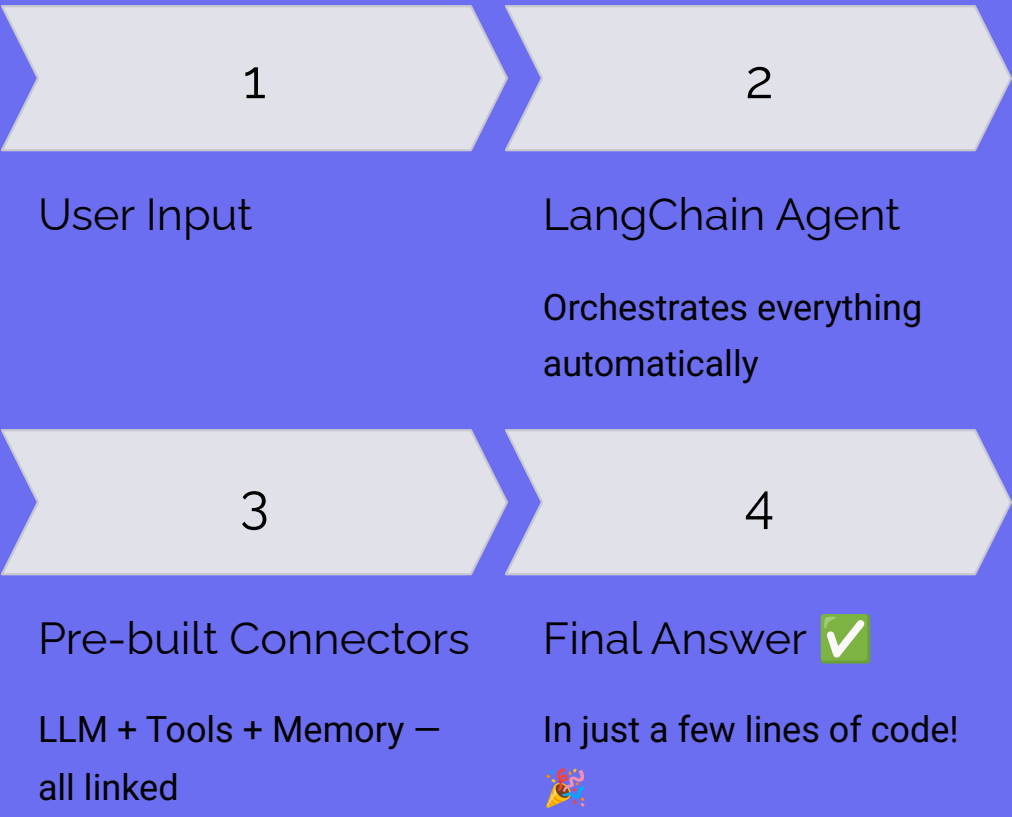
WITHOUT LANGCHAIN

You write **lots of custom code** to wire everything together manually:



WITH LANGCHAIN

Ready-made building blocks — **snap and go!**



Real Example: "What is the weather in Pune?"



LangChain coordinates every step — you just write the goal!

🔑 **Key Point:** LangChain does **NOT replace the LLM**. It helps you build a complete application **AROUND** the LLM — connecting it to tools, memory, and logic.

LangChain: Your First Chain (Code)

Step 1 — Install

```
pip install langchain openai
```

Step 2 — Build a Simple Chain

```
from langchain.llms import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate

# Define a prompt template
prompt = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple English."
)

# Create a chain
llm = OpenAI(temperature=0.7)
chain = LLMChain(llm=llm, prompt=prompt)

# Run it!
result = chain.run("machine learning")
print(result)
```

How it Works

1 Define a Prompt Template

Tell the LLM *how* to respond using a template with placeholders.

2 Create a Chain

Connect the prompt to your chosen LLM model.

3 Run & Get Output

Pass in your topic and receive a clear, simple explanation instantly.

Dr. Anilkumar V. Brahmane / Computer Engineering, Sanjivani College of Engineering

LangChain Tools: Giving AI "Hands"

What Tools Do

Tools let the agent **do real things** in the world — not just talk. LangChain ships with **50+ built-in tools**: Wikipedia, Calculator, Weather, SQL, and more.

```
from langchain.agents import load_tools, initialize_agent
from langchain.llms import OpenAI
```

```
llm = OpenAI(temperature=0)
tools = load_tools(["wikipedia", "llm-math"], llm=llm)
```

```
agent = initialize_agent(
    tools, llm,
    agent="zero-shot-react-description"
)
agent.run(
    "What is the square root of the year Python was created?"
)
```



Search

Browse Wikipedia, Google, or the web live



Calculate

Solve math problems with a built-in calculator



Query Data

Read from SQL databases and files



Call APIs

Interact with external services and tools

What is AutoGPT? 🚀

Auto + GPT = Using GPT-like language models to perform tasks **more autonomously** — planning, acting, and looping without constant human guidance.



Normal ChatGPT

You guide every step manually:

1. You: "Write a Python program"
2. AI gives code
3. You: "Now test it"
4. AI helps test
5. You: "Fix this bug..."



Autonomous Agent (AutoGPT)

You give ONE goal — agent does the rest:

1. Goal: "Create a Python adventure game"
2. Create plan
3. Write code
4. Check code → Find problems
5. Modify → Test → Improve
6. ✅ Final result



The Analogy

LangChain = you drive the car.

AutoGPT = self-driving car — you just say the destination!



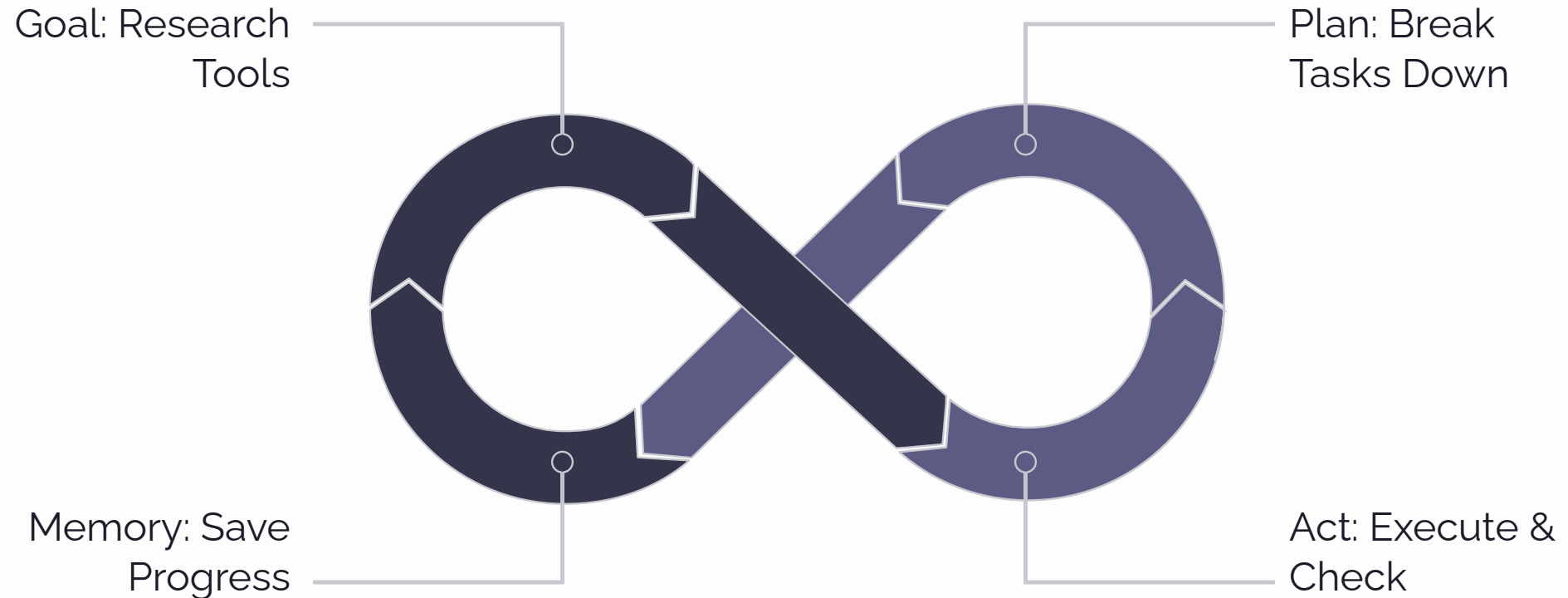
What It Can Do

Browse the web, write code, save files, and **loop until the goal is done**.



Note: AutoGPT is best understood as an **EXAMPLE** of autonomous-agent systems — not something you must always use.

AutoGPT: How It Works (Step by Step)



```
# AutoGPT-style loop (simplified concept)
goal = "Research top AI tools in 2024"
tasks = agent.plan(goal)    # Break into steps
for task in tasks:
    result = agent.execute(task) # Do each step
    agent.memory.save(result)    # Remember it
```

What is CrewAI? 👥 Multi-Agent Systems



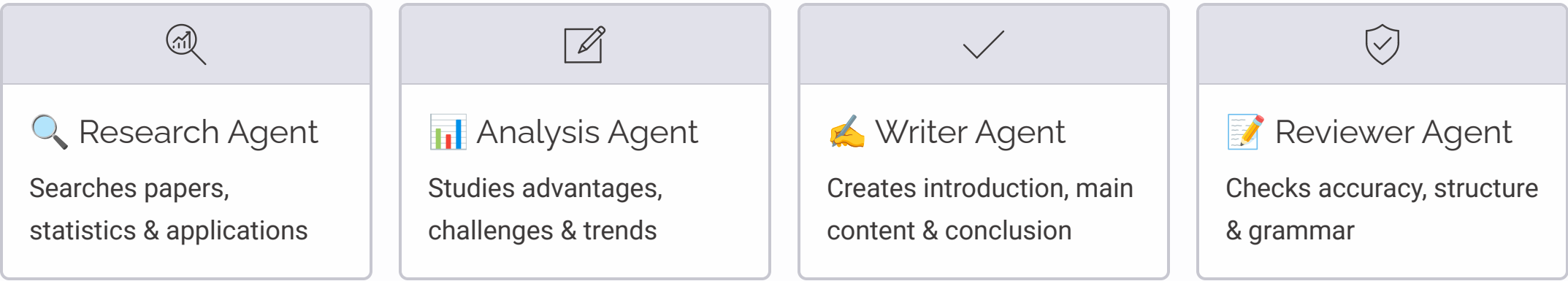
Imagine you have **one AI assistant doing everything** vs a **TEAM of AI agents** — each with a specialized role, working together like a real office crew!

 Single Agent One AI does <i>everything</i> : search, analyze, write, review	 Multi-Agent Team Each agent has <i>one specific job</i> — faster, smarter, better
---	---

CrewAI Flow



🎯 Real-World Example: "Prepare a Report on Generative AI in Education"



🟢 ☒ **Result:** A complete, well-researched, well-written, and reviewed report — produced by a coordinated AI team!

CrewAI: Build Your First Crew (Code)

```
from crewai import Agent, Task, Crew

# Define agents with roles
researcher = Agent(
    role="Researcher",
    goal="Find the latest AI trends",
    backstory="Expert at finding information online"
)

writer = Agent(
    role="Writer",
    goal="Write a blog post from research",
    backstory="Skilled technical writer"
)

# Define tasks
task1 = Task(
    description="Research top AI trends 2024",
    agent=researcher
)
task2 = Task(
    description="Write a blog post from the research",
    agent=writer
)

# Create and run the crew
crew = Crew(
    agents=[researcher, writer],
    tasks=[task1, task2]
)
result = crew.kickoff()
print(result)
```

How the Crew Flows

Dr. Anilkumar V. Brahmane / Computer Engineering, Sanjivani College of Engineering

- 1

1. Define Agents

Give each agent a role, goal, and backstory
- 2

2. Assign Tasks

Map specific tasks to the right agent
- 3

3. Kickoff Crew

Agents collaborate and pass results to each other

Memory in Agents: Short-Term Memory 🧠 ⚡

What is Short-Term Memory?

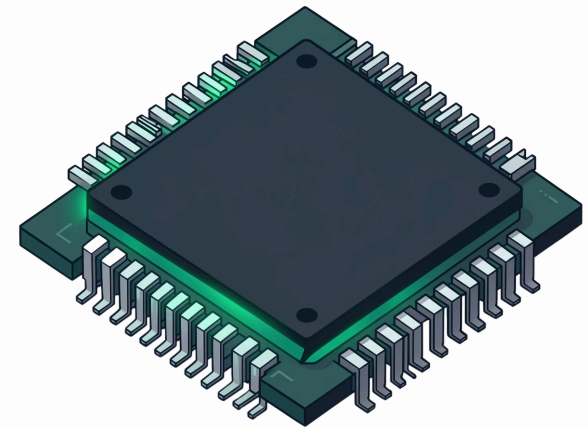
Short-term memory = what the agent remembers **during one conversation** – like RAM in a computer. When the chat ends, it's gone! The agent forgets everything.

In LangChain, this is called **ConversationBufferMemory**.

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain.llms import OpenAI

memory = ConversationBufferMemory()
chat = ConversationChain(
    llm=OpenAI(),
    memory=memory
)

chat.predict(input="My name is Anil")
chat.predict(input="What is my name?")
# It remembers! Output: "Your name is Anil"
```

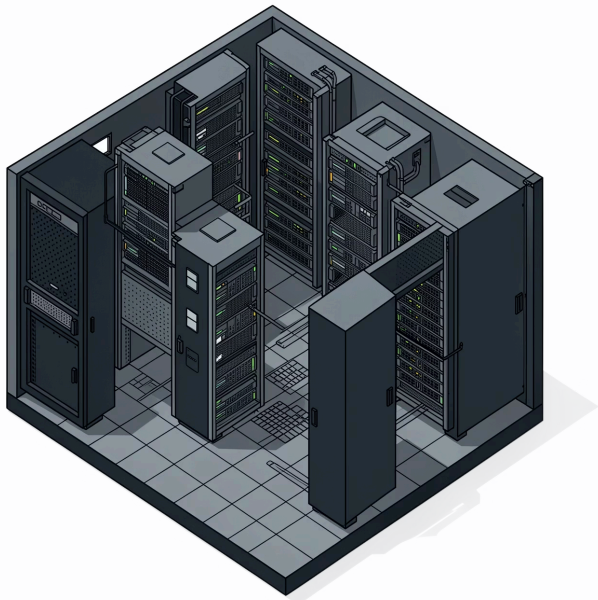


Memory in Agents: Long-Term Memory

What is Long-Term Memory?

Long-term memory = stored in a **database or vector store** — persists across sessions like a hard drive. Agent can recall past conversations, user preferences, and facts from days ago.

Popular Tools: FAISS · Chroma · Pinecone



```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
```

```
# Save facts to long-term memory
vectorstore = Chroma(
    embedding_function=OpenAIEmbeddings()
)
vectorstore.add_texts([
    "User prefers Python",
    "User is from Pune"
])
```

```
# Retrieve later
docs = vectorstore.similarity_search(
    "What does the user prefer?"
)
print(docs[0].page_content)
# Output: "User prefers Python"
```

Short-Term vs Long-Term Memory: Quick Comparison

Feature	Short-Term Memory ⚡	Long-Term Memory 💾
Scope	Current conversation	Information saved for future use
Duration	Temporary – lost when session ends	Persistent – survives restarts & sessions
Storage	RAM / in-memory (conversation buffer)	Database / vector store (FAISS, Chroma, Pinecone)
Context	Recent context only	Historical information across sessions
Scale	Limited to current chat	Millions of facts & records
Analogy	Working memory (active thinking)	Stored memory (long-term recall)
Use Case	Active conversation context	User history & knowledge base across tasks

🤔 Why Do Agents Need Memory?

❌ Without Memory

Every new interaction starts with **no context** – the agent forgets everything from previous sessions, leading to repetitive, impersonal responses.

✅ With Memory

Agent uses **previous information** → Better, more personalized decisions. Remembers preferences, history, and past interactions.

👨‍🎓 Human Analogy

Teaching a student CNN today = Short-Term Memory
The student understands it right now, in the current session.

Student remembers CNN 6 months later = Long-Term Memory
The knowledge was stored and can be recalled across time.

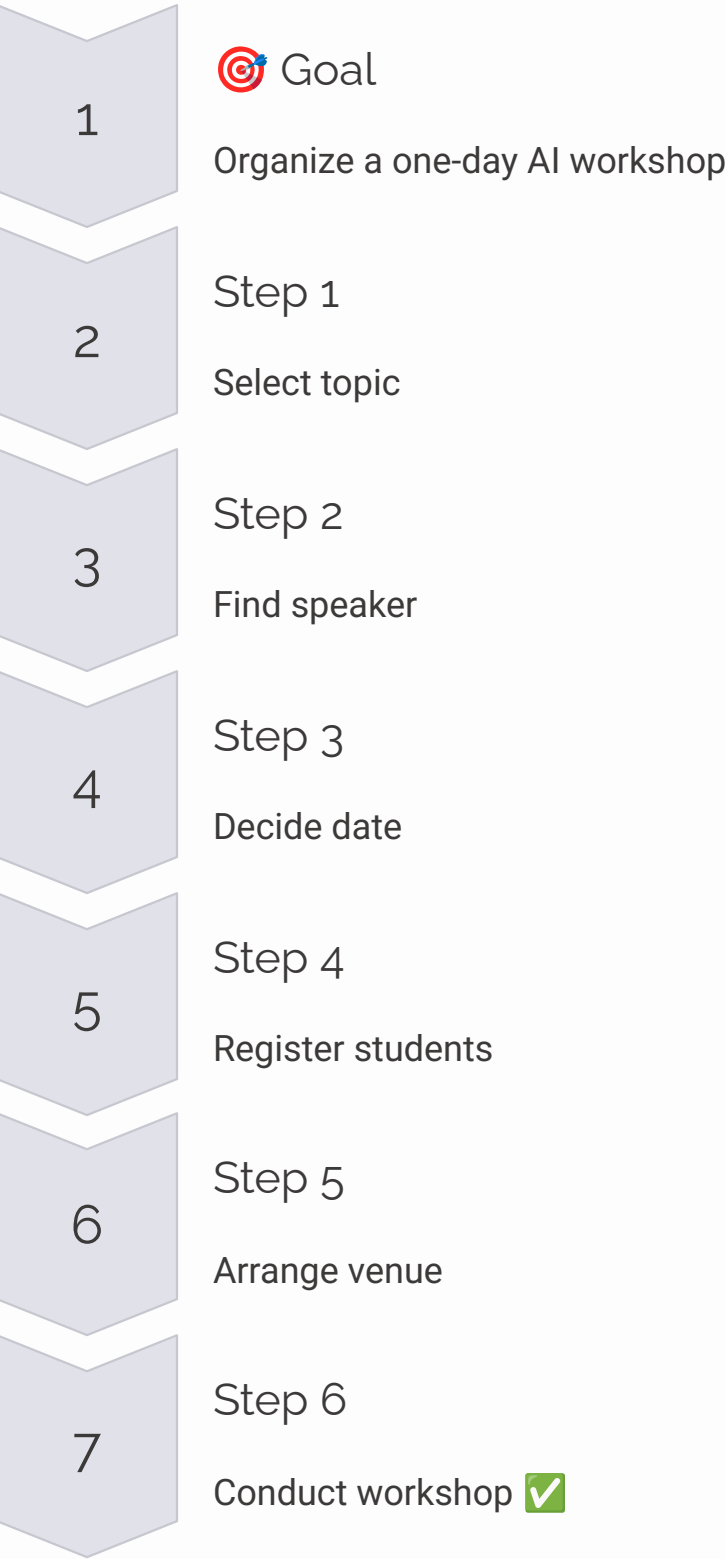
📄💡 **Best Practice:** Use **BOTH!** Short-term for the current chat, long-term for user history & knowledge base – together they make agents truly intelligent.

Planning in AI Agents

Planning = the agent **breaks a big goal into smaller steps** before acting — like making a to-do list before starting work.

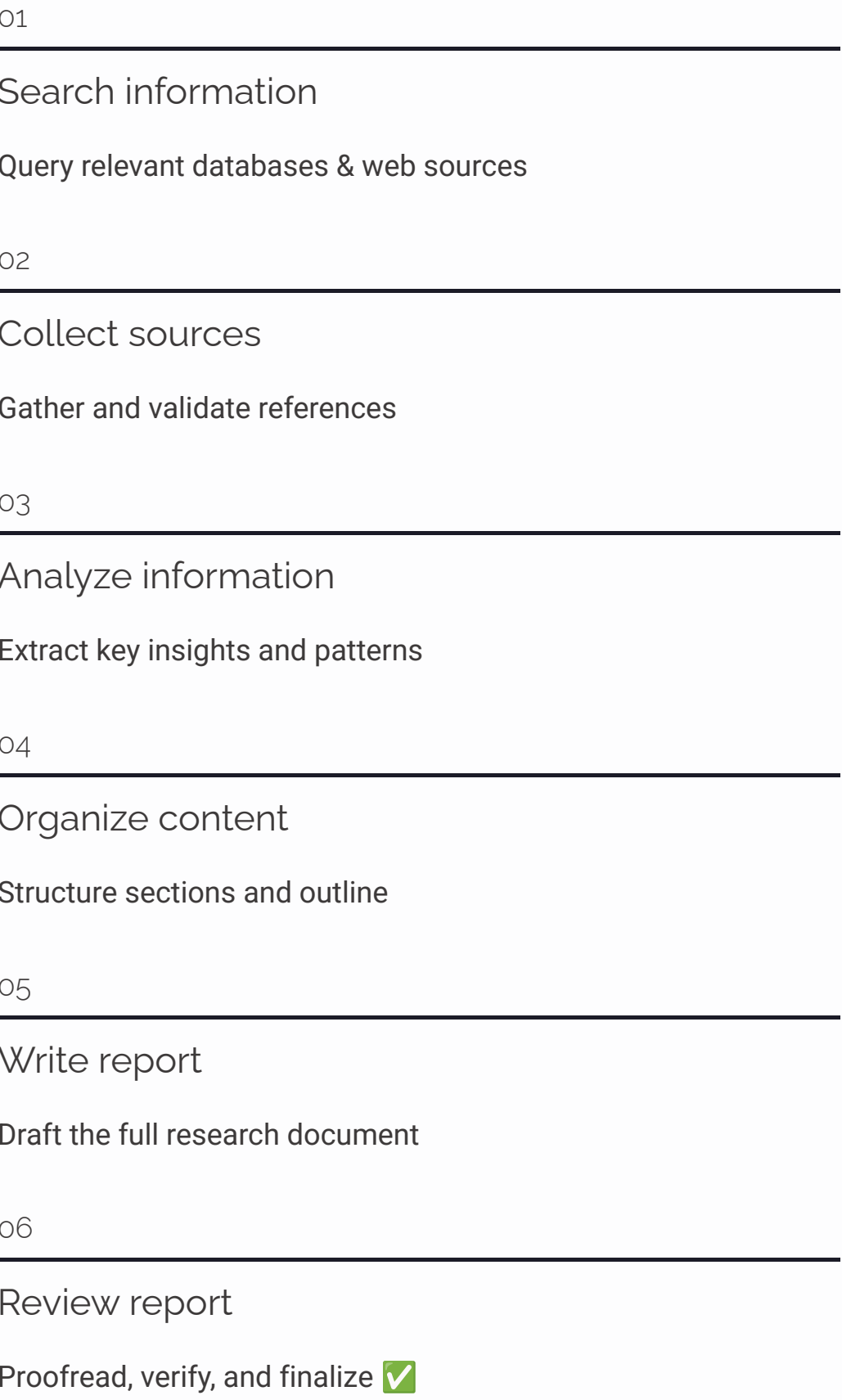
Workshop Analogy

"Suppose I tell you: **Organize a one-day AI workshop**. You don't immediately start randomly doing things. You make a plan first."



Agent Planning Example

Goal: "**Prepare a research report on Generative AI**"



ReAct Pattern (Reason + Act)

The most popular planning technique — the agent **thinks out loud**, then takes action based on its reasoning.

ReAct pattern — agent thinks before acting

Thought: I need to find the population of India
Action: Search["population of India 2024"]
Observation: 1.44 billion
Thought: Now I have the answer
Final Answer: India's population is 1.44 billion

```
from langchain.agents import AgentType,
initialize_agent
agent = initialize_agent(
    tools, llm,
    agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
    verbose=True
)
```

Planning vs Reasoning

 Planning

"What steps should I take?"
Deciding the sequence of actions needed to reach the goal.

 Reasoning

"Which step/action makes sense and WHY?"
Justifying each decision with logic and context.

 Pune → Mumbai Travel Example

Planning: Check transport options → Choose bus/train → Book ticket → Travel

Reasoning: "The train is better because traffic may be heavy on the expressway today."

Reasoning: How Agents Think 🤔

Reasoning helps the agent decide: *"WHY should I do it this way?"* It's the process of determining the appropriate answer/action.

🧮 Simple Reasoning Example

Goal: **"Find the average of 10, 20 and 30."**

01

Add the numbers

$$10 + 20 + 30 = \mathbf{60}$$

02

Divide by count

$$60 \div 3 = \mathbf{20}$$

03

Result

Average = **20** ✅

Chain-of-Thought (CoT) 🔗

Agent **explains its reasoning step by step** — like showing your math work. Adding **"Think step by step"** to your prompt dramatically improves accuracy!

Chain-of-Thought prompting example
prompt = ""
Q: A train travels 60 km/h for 2.5 hours.
How far does it go? Think step by step:

Step 1: Speed = 60 km/h, Time = 2.5 hours
Step 2: Distance = Speed × Time = 60 × 2.5
Step 3: Distance = 150 km

Answer: 150 km
""

🔗 Chain-of-Thought (CoT)

One linear path of reasoning, step by step. Like showing your work in an exam.

🌳 Tree-of-Thought (ToT)

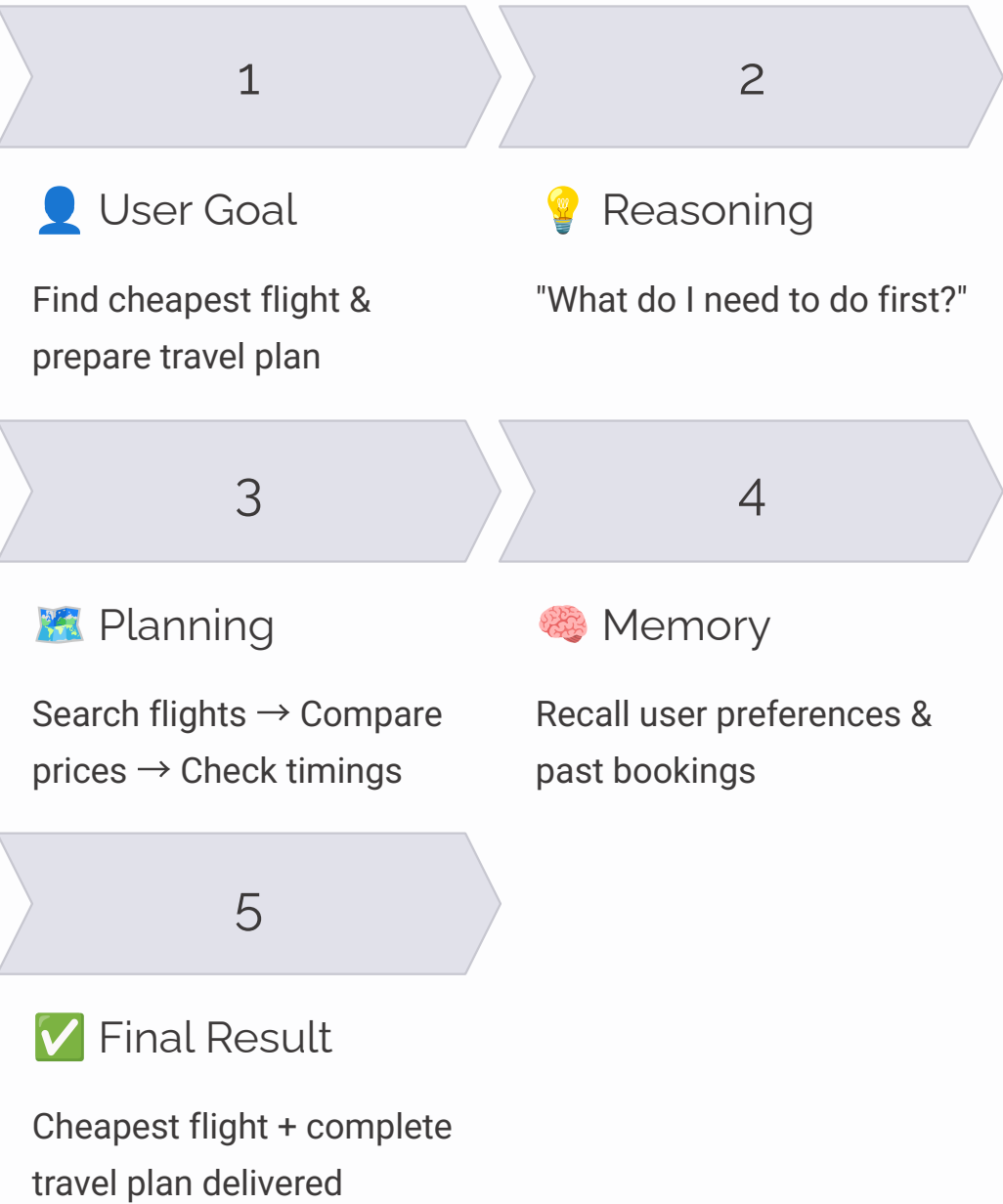
Agent explores **multiple solution paths** and picks the best one. Better for complex problems.

💡 Pro Tip

Always add **"Think step by step"** to prompts — it's the simplest way to boost agent accuracy!

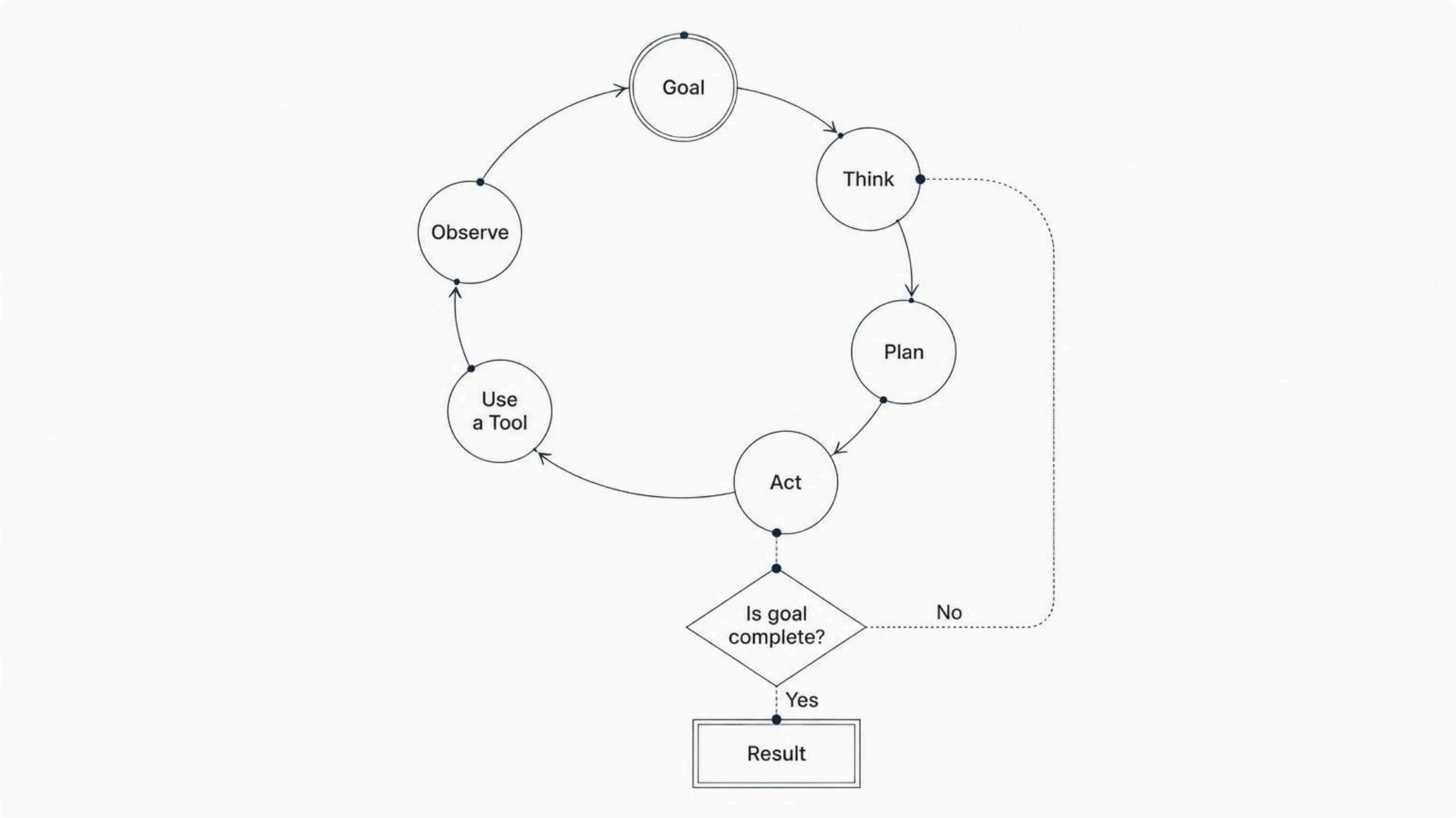
🔄 Planning + Reasoning + Tools Together

Goal: **"Find the cheapest flight and prepare my travel plan."**



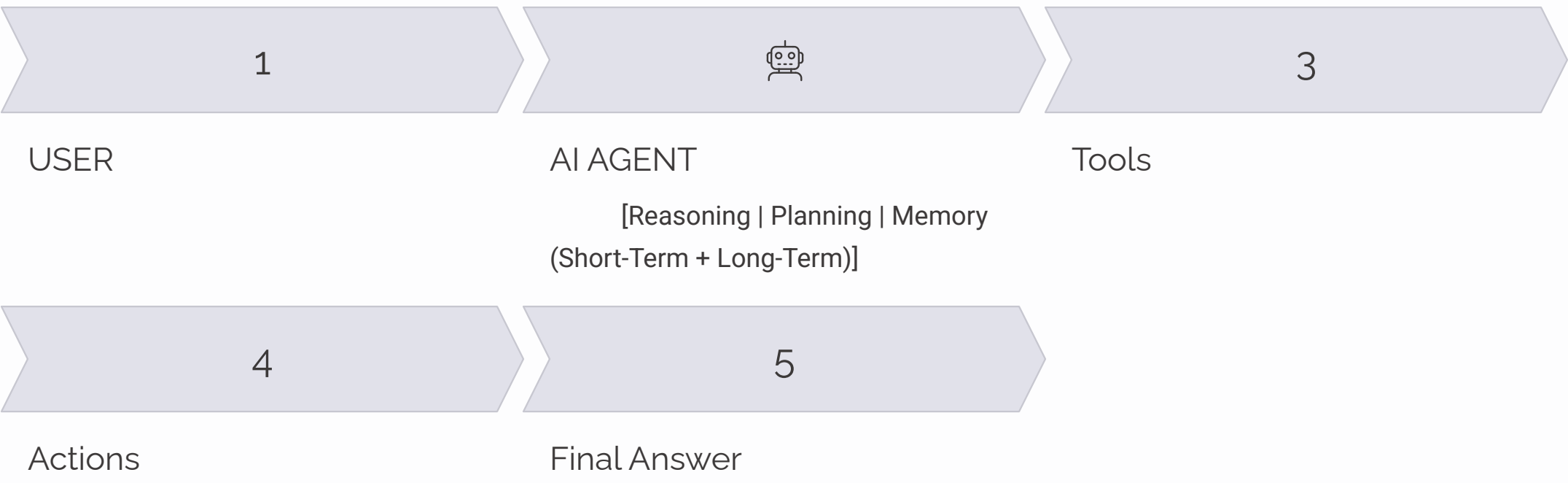
The Agent Loop — One Very Important Concept

Many agent systems work in a loop — the agent doesn't do everything in one step. It can: Think → Act → Observe → Think again → Act again, until the task is completed.



📌 This loop is the heart of every autonomous agent — it keeps going until the job is done!

AI Agent Architecture



Multi-Agent Systems



LangChain helps you build LLM-powered applications and agents, CrewAI helps you build teams of agents, AutoGPT represents autonomous goal-oriented agent behavior, memory lets agents retain information, and planning + reasoning help agents decide what to do and how to achieve a goal.

Your Learning Roadmap

Follow this **5-level path** to master AI Agents – from fundamentals to autonomous systems.

1

 **Level 5 — Advanced**
Agentic RAG · Multi-agent collaboration · Evaluation · Guardrails · Human-in-the-loop · Autonomous agents

2

 **Level 4 — Frameworks**
LangChain · LangGraph · CrewAI · Multi-agent systems

3

 **Level 3 — Memory**
Short-term · Long-term · Vector databases · Embeddings · Retrieval

4

 **Level 2 — AI Agents**
Agent workflow · Tools · Planning · Reasoning

5

 **Level 1 — Foundation**
What is an LLM? Generative AI · Prompt · API · Tool

Quick Start: 4-Week Plan

1

Week 1
Install LangChain → Build your first chain
→ Add tools

2

Week 2
Add memory → Build a chatbot that
remembers you

3

Week 3
Try CrewAI → Build a 2-agent research +
writing crew

4

Week 4
Explore AutoGPT → Give it a real-world goal
and watch it work!

Easy Analogy Table

AI Concept	Human Analogy
LLM	 Brain
Agent	 Smart employee
Tool	 Computer / phone / software
LangChain	 Toolbox for building workflow
CrewAI	 Team of employees
AutoGPT	 Employee working autonomously
Short-term memory	 Remembers current meeting
Long-term memory	 Employee's stored records
Planning	 Making a to-do list
Reasoning	 Deciding what makes sense

  **Start right now:** `pip install langchain crewai openai chromadb`